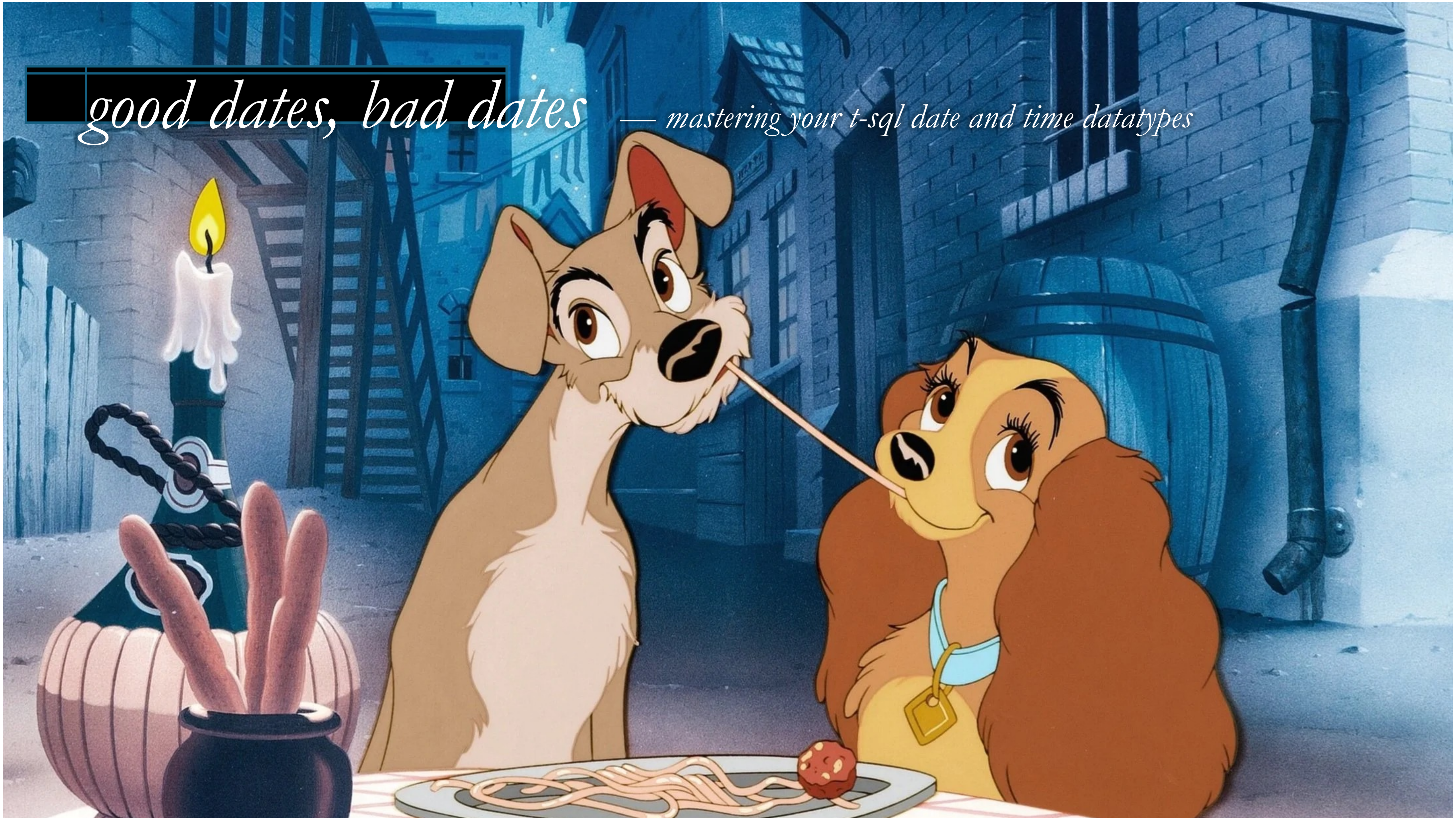
good dates, bad dates — *mastering your t-sql date and time datatypes*



date math is hard.

Table of months

Months (Roman)	Lengths before 46 BC	Length in 46 BC	Lengths as of 45 BC	Months (English)
Ianuarius[3]	29	29	31	January
Februarius	28 (in common years) In intercalary years: 23 if Intercalaris is variable 23–24 if Intercalaris is fixed	28	28 (leap years: 29)	February
Intercalaris (Mercedonius) (only in intercalary years)	27 (or possibly 27–28)	23	—	—
Martius	31	31	31	March
Aprilis	29	29	30	April
Maius	31	31	31	May
Iunius[3]	29	29	30	June
Quintilis[4] (Iulius)	31	31	31	July
Sextilis (Augustus)	29	29	31	August
September	29	29	30	September
October	31	31	31	October
2nd intercalary	—	33	—	—
3rd intercalary	—	34	—	—
November	29	29	30	November
December	29	29	31	December
Total	355 or 377–378	445	365–366	365–366

The Julian Calendar was kind of high-maintenance.

date math is hard.

It was pretty complex, but even worse, it
assumed that a year was 365.25 days long.

Spoiler: it's 365.2422

date math is hard.

It was pretty complex, but even worse, it assumed that a year was 365.25 days long.

Spoiler: it's 365.2422

So, the Julian calendar *gained* one day every 128 years, or 3.1 days every 400 years.

In 1582 pope Gregorius XIII reformed the calendar by introducing the Gregorian calendar, which is what we use today.

date math is hard.

Just 170 years later, Britain adopted the
Gregorian calendar with the 1752 Calendar Act.

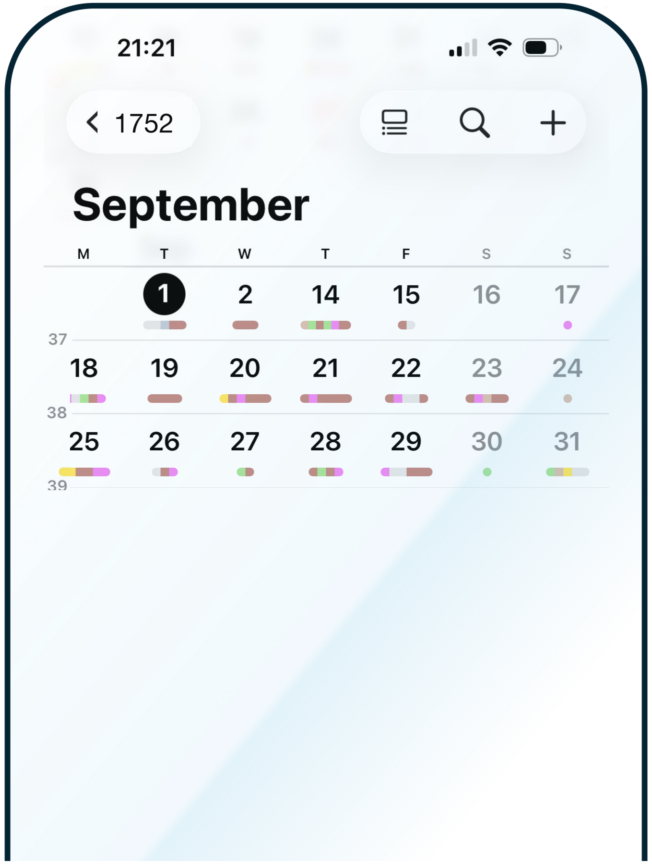
date math is hard.

Just 170 years later, Britain adopted the
Gregorian calendar with the 1752 Calendar Act.

By then, the calendar had drifted another 11 days.

date math is hard.

So they deleted 11 days from September.



date math is hard.

No seriously, if you're on macOS or Linux, try it yourself:

```
# cal 9 1752
```

date math is hard.

No seriously, if you're on macOS or Linux, try it yourself:

```
# cal 9 1752
 September 1752
Su Mo Tu We Th Fr Sa
       1  2 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30



# █
```

date math is hard.

To be continued.

daniel hutmacher

SQL Server developer since 1999
Database consultant
Conference organizer, speaker, blogger
Microsoft Data Platform MVP

@dhma.ch
/in/danielhutmacher/

daniel@strd.co

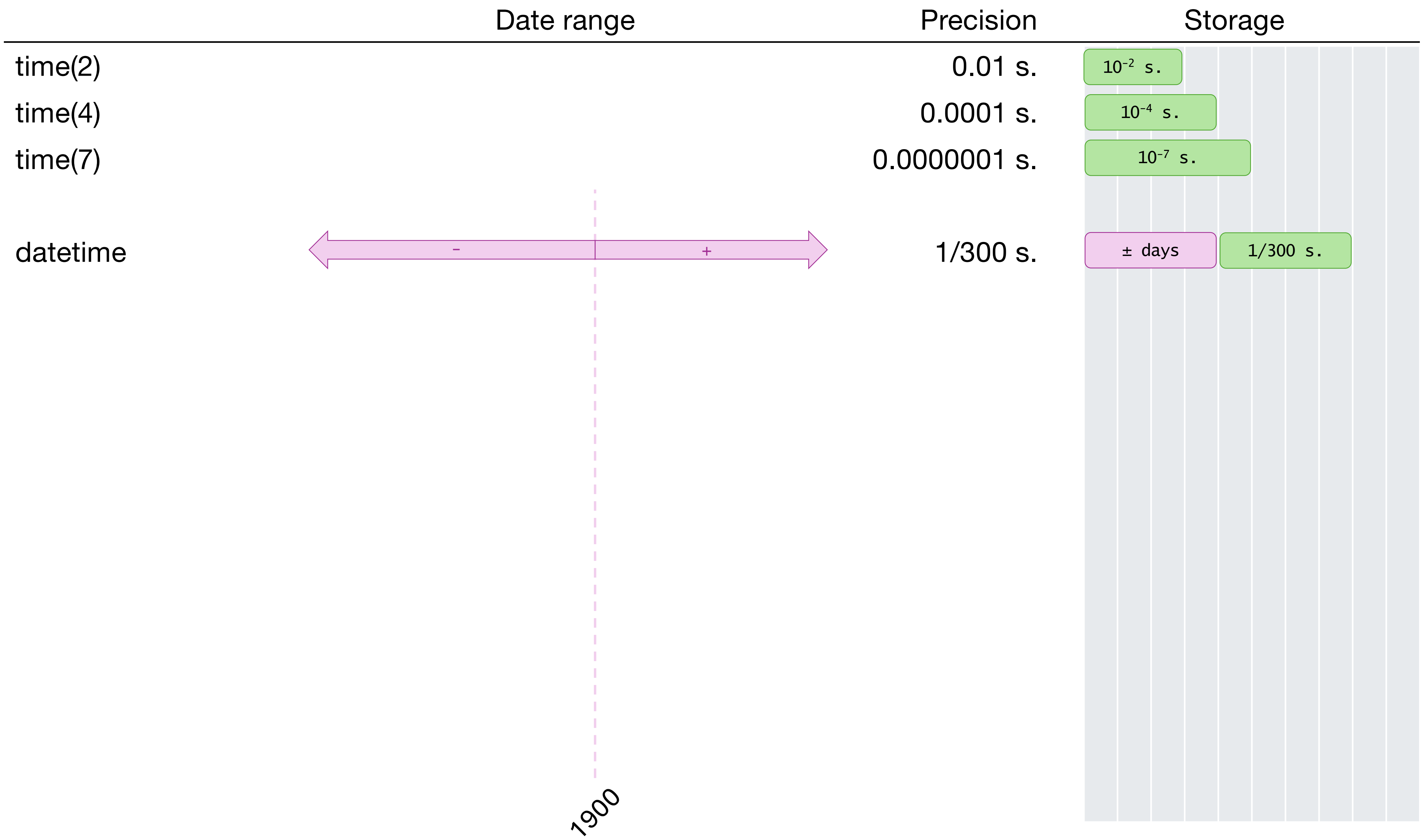
who's paying?

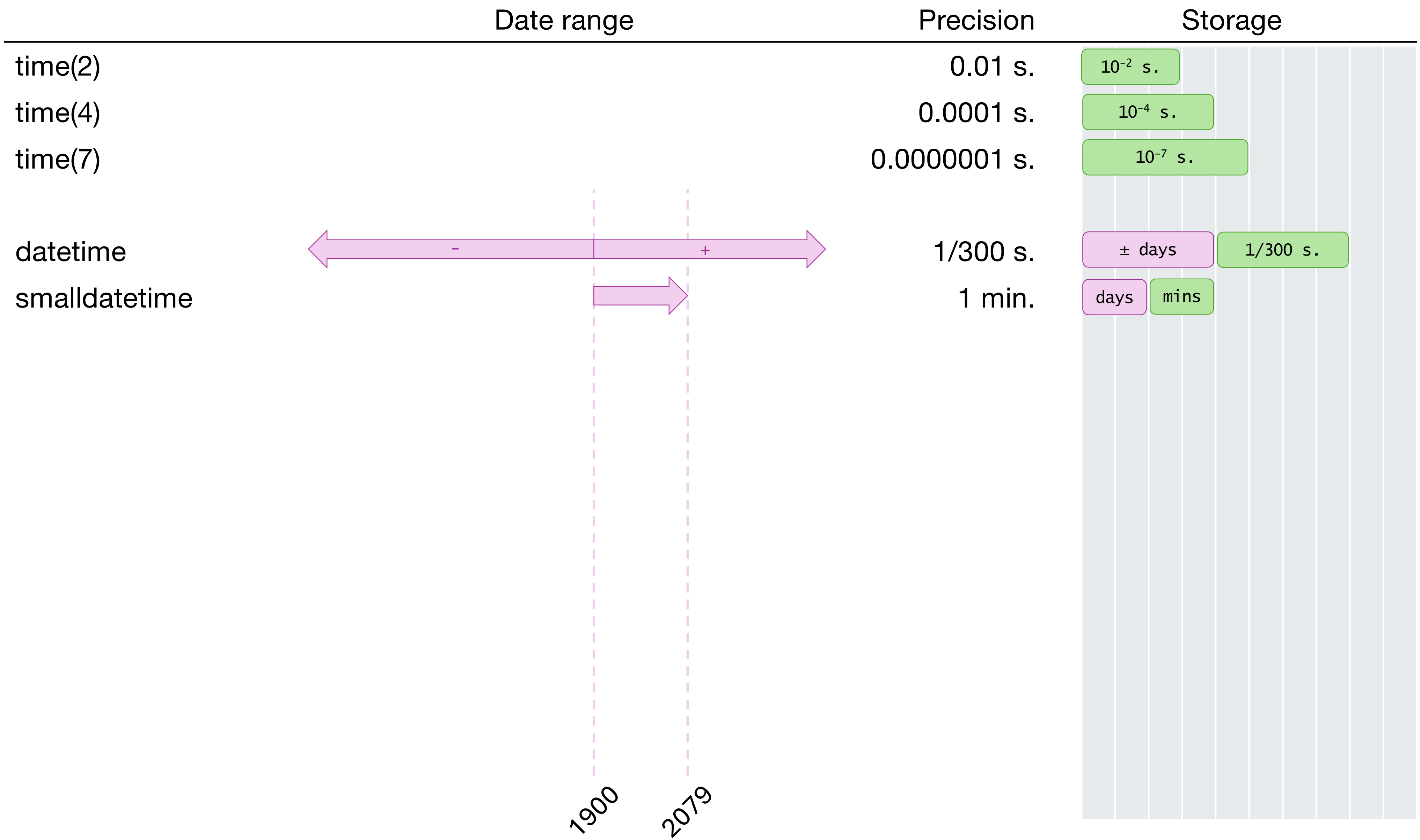
know your type

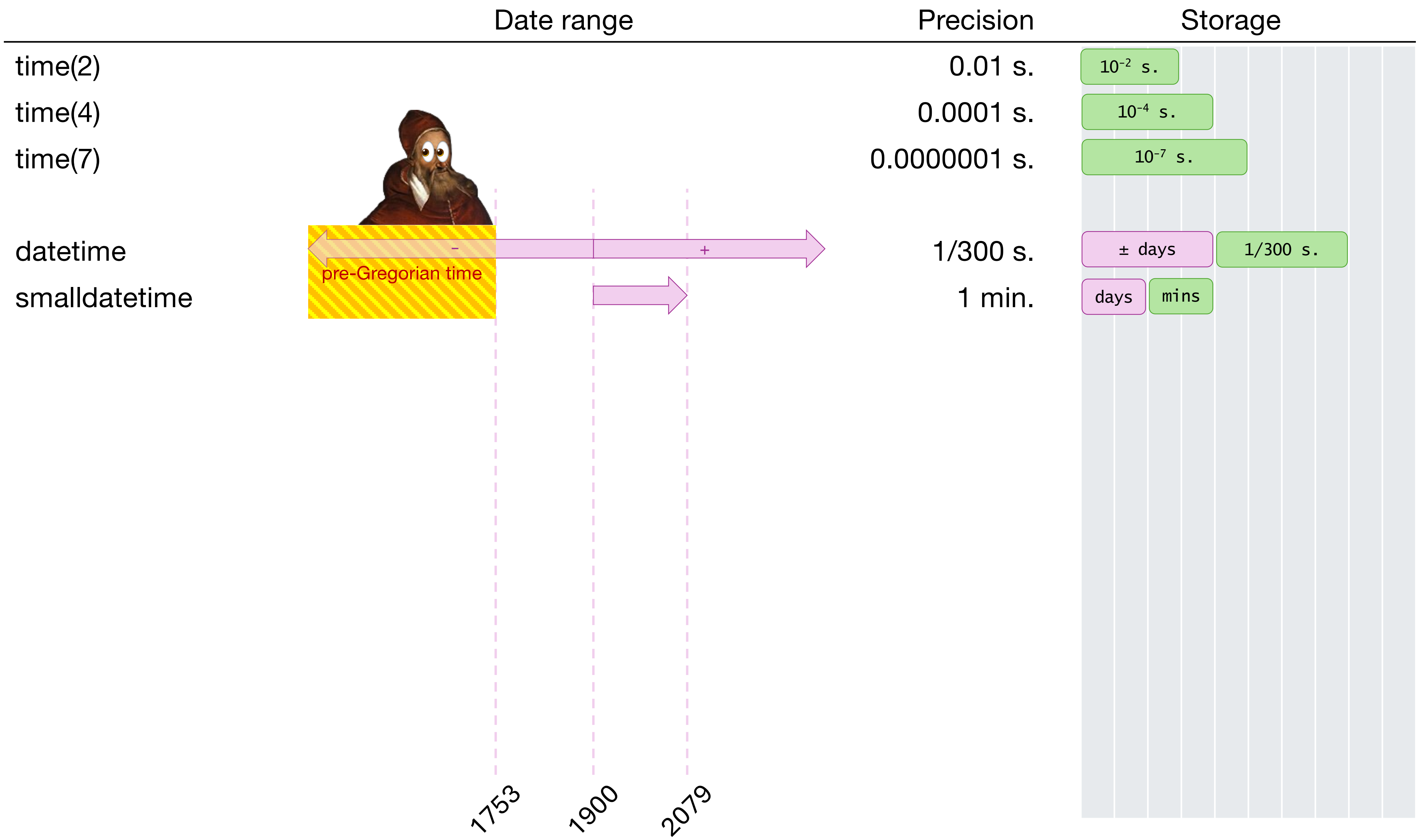
	Precision	Storage
time(0)	1 s.	1 s.

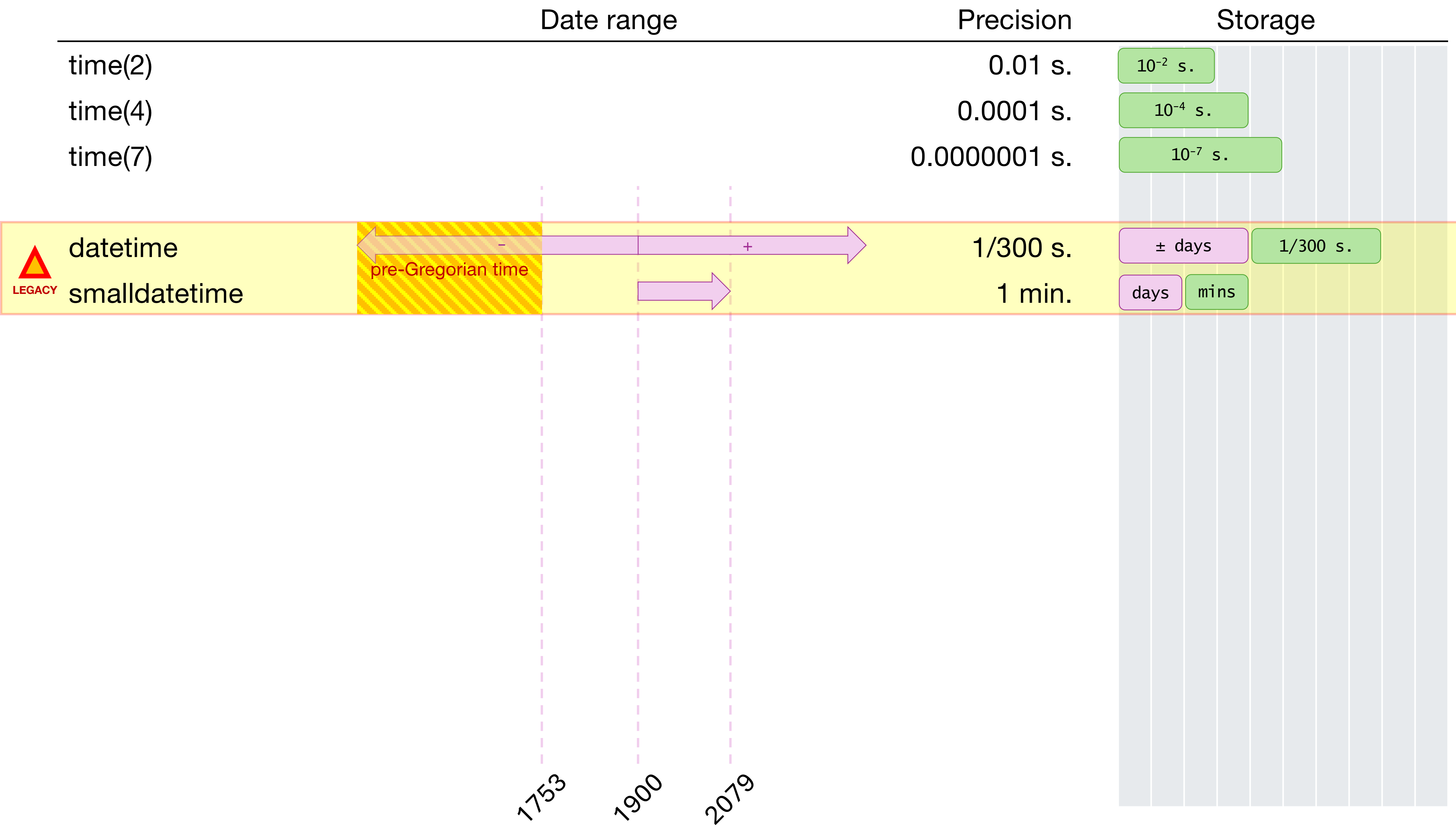
	Precision	Storage
time(0)	1 s.	1 s.
time(1)	0.1 s.	10^{-1} s.
time(2)	0.01 s.	10^{-2} s.
time(3)	0.001 s.	10^{-3} s.
time(4)	0.0001 s.	10^{-4} s.
time(5)	0.00001 s.	10^{-5} s.
time(6)	0.000001 s.	10^{-6} s.
time(7)	0.0000001 s.	10^{-7} s.

	Precision	Storage
time(2)	0.01 s.	10^{-2} s.
time(4)	0.0001 s.	10^{-4} s.
time(7)	0.0000001 s.	10^{-7} s.

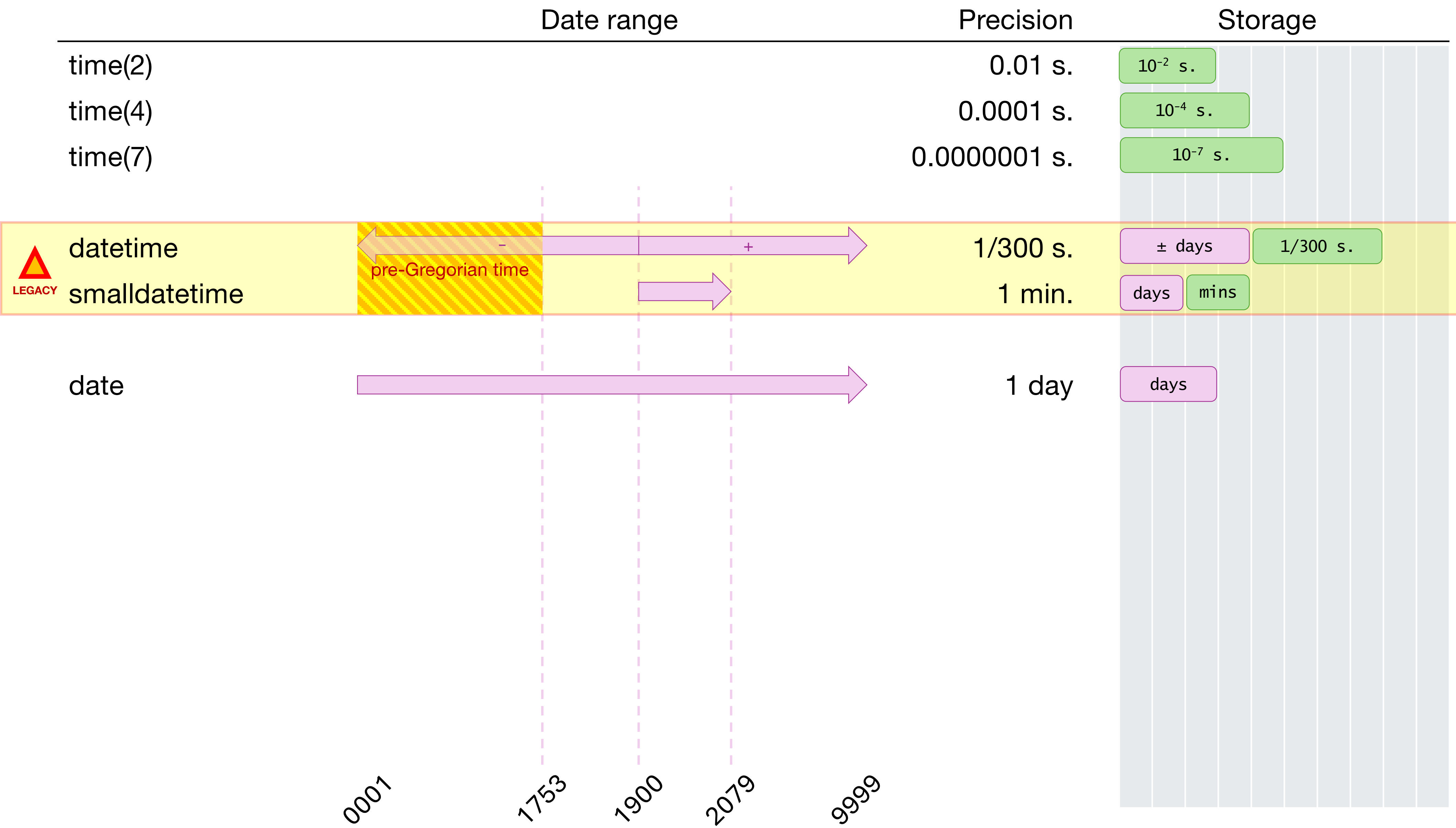

Date range
Precision
Storage
time(2)
0.01 s.
10-2 s.
time(4)
0.0001 s.
10-4 s.
time(7)
0.0000001 s.
10-7 s.
datetime
−
+
1/300 s.
± days
1/300 s.
1900

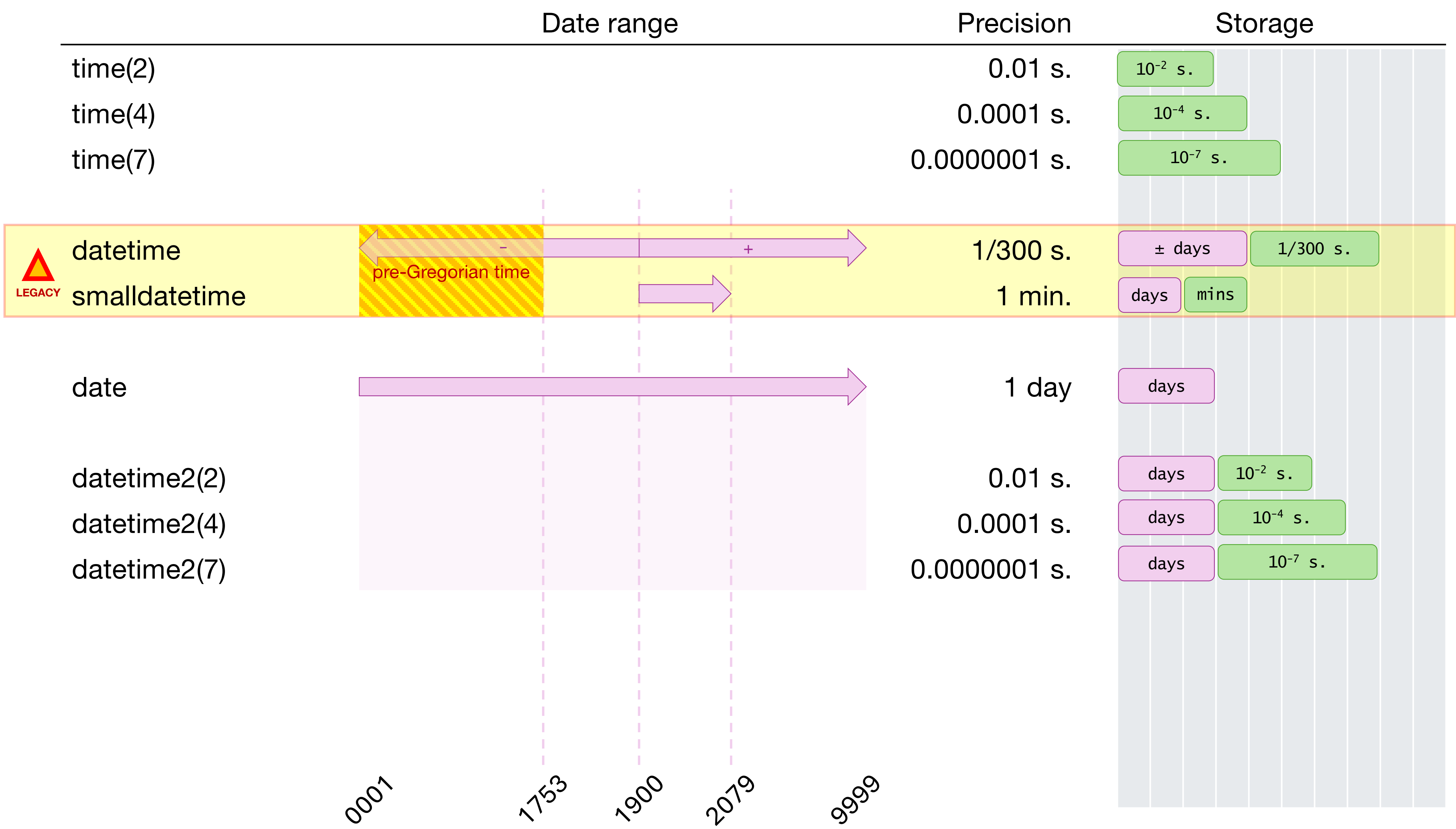

Date range
Precision
Storage
time(2)
0.01 s.
10^{-2} s.
time(4)
0.0001 s.
10^{-4} s.
time(7)
0.0000001 s.
10^{-7} s.
datetime
−
+
1/300 s.
± days
1/300 s.
smalldatetime
1 min.
days
mins
1900
2079


Date range
Precision
Storage
time(2)
0.01 s.
10-2 s.
time(4)
0.0001 s.
10-4 s.
time(7)
0.0000001 s.
10-7 s.
datetime
-
+
pre-Gregorian time
1/300 s.
± days
1/300 s.
smalldatetime
1 min.
days
mins
1753
1900
2079

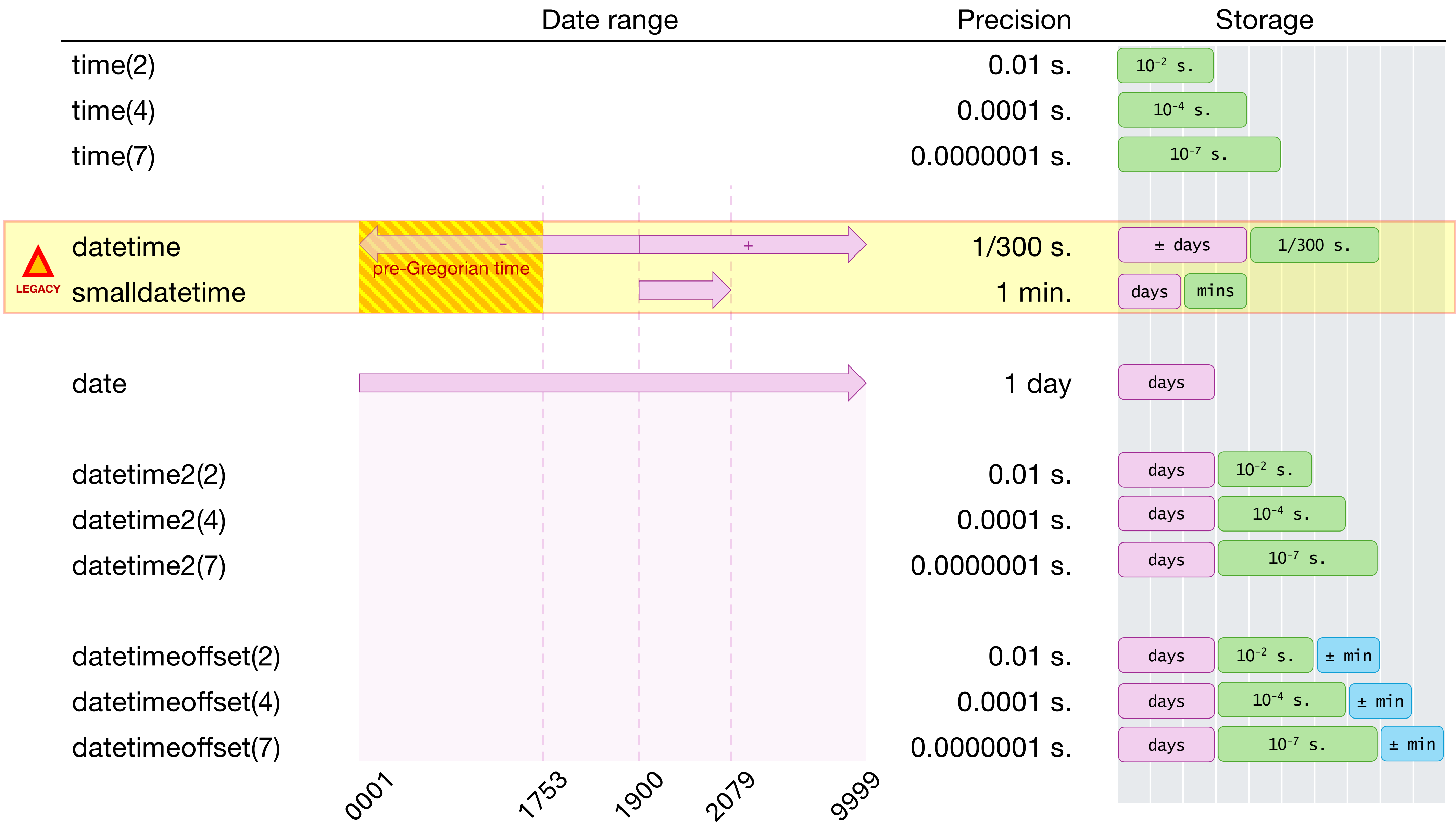


Date range
Precision
Storage
time(2)
0.01 s.
10⁻² s.
time(4)
0.0001 s.
10⁻⁴ s.
time(7)
0.0000001 s.
10⁻⁷ s.
LEGACY
datetime
–
+
pre-Gregorian time
1/300 s.
± days
1/300 s.
smalldatetime
1 min.
days
mins
1753
1900
2079


Date range
Precision
Storage
time(2)
0.01 s.
10^-2 s.
time(4)
0.0001 s.
10^-4 s.
time(7)
0.0000001 s.
10^-7 s.
LEGACY
datetime
−
+
pre-Gregorian time
1/300 s.
± days
1/300 s.
smalldatetime
1 min.
days
mins
date
1 day
days
0001
1753
1900
2079
9999



Date range
Precision
Storage
time(2)
0.01 s.
10^-2 s.
time(4)
0.0001 s.
10^-4 s.
time(7)
0.0000001 s.
10^-7 s.
LEGACY
datetime
–
+
pre-Gregorian time
1/300 s.
± days
1/300 s.
smalldatetime
1 min.
days
mins
date
1 day
days
datetime2(2)
0.01 s.
days
10^-2 s.
datetime2(4)
0.0001 s.
days
10^-4 s.
datetime2(7)
0.0000001 s.
days
10^-7 s.
0001
1753
1900
2079
9999


Date range
Precision
Storage
time(2)
0.01 s.
10^-2 s.
time(4)
0.0001 s.
10^-4 s.
time(7)
0.0000001 s.
10^-7 s.
LEGACY
datetime
-
+
pre-Gregorian time
1/300 s.
± days
1/300 s.
smalldatetime
1 min.
days
mins
date
1 day
days
datetime2(2)
0.01 s.
days
10^-2 s.
datetime2(4)
0.0001 s.
days
10^-4 s.
datetime2(7)
0.0000001 s.
days
10^-7 s.
datetimeoffset(2)
0.01 s.
days
10^-2 s.
± min
datetimeoffset(4)
0.0001 s.
days
10^-4 s.
± min
datetimeoffset(7)
0.0000001 s.
days
10^-7 s.
± min
0001
1753
1900
2079
9999

know your type

takeaways

- Avoid datetime and smalldatetime for new development

know your type

takeaways

- Avoid datetime and smalldatetime for new development
- Do not depend on the local server timezone

know your type

takeaways

- Avoid datetime and smalldatetime for new development
- Do not depend on the local server timezone
- In an international environment, save everything in UTC

know your type

takeaways

- Avoid datetime and smalldatetime for new development
- Do not depend on the local server timezone
- In an international environment, save everything in UTC
- … or use the datetimeoffset type

functions

functions

DATEPART(datepart , date) = number

functions

```
DATEPART(datepart, date) = number
```

```
DATEPART(month, '2026-08-12') =
```

functions

```
DATEPART(datepart, date) = number
```

```
DATEPART(month, '2026-08-12') = 8
```

functions

```
DATEPART(datepart, date) = number
```

```
DATEPART(month, '2026-08-12') = 8

DATEPART(iso_week, '2026-08-26') = 35
```

functions

DATEPART(datepart , date) = number

YEAR(date) = DATEPART(year, date)
MONTH(date) = DATEPART(month, date)
DAY(date) = DATEPART(day, date)

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

functions

```
DATEPART(datepart, date) = number

DATENAME(datepart, date) = name
```

```
DATENAME(month, '2026-08-04') =
```

functions

```
DATEPART(datepart, date) = number

DATENAME(datepart, date) = name
```

```
DATENAME(month, '2026-08-04') = 'August'
```

functions

```
DATEPART(datepart, date) = number

DATENAME(datepart, date) = name
```

```
DATENAME(month, '2026-08-04') = 'August'

DATENAME(weekday, '2026-08-04') = 'Tuesday'
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

functions

```
DATEPART( datepart , date ) = number

DATENAME( datepart , date ) = name

DATEADD( datepart , count , from date ) = to date

DATEDIFF( datepart , from date , to date ) = count
```

```
DATEDIFF(day, '2026-08-30', '2026-09-03') =
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

```
DATEDIFF(day, '2026-08-30', '2026-09-03') = 4
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

```
DATEDIFF(day, '2026-08-30', '2026-09-03') = 4

DATEDIFF(month, '2026-07-01', '2026-07-31') =
```

functions

```
DATEPART( datepart , date ) = number

DATENAME( datepart , date ) = name

DATEADD( datepart , count , from date ) = to date

DATEDIFF( datepart , from date , to date ) = count
```

```
DATEDIFF(day, '2026-08-30', '2026-09-03') = 4

DATEDIFF(month, '2026-07-01', '2026-07-31') = 0
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

```
DATEDIFF(day, '2026-08-30', '2026-09-03') = 4

DATEDIFF(month, '2026-07-01', '2026-07-31') = 0

DATEDIFF(month, '2026-07-31', '2026-08-01') =
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

```
DATEDIFF(day, '2026-08-30', '2026-09-03') = 4

DATEDIFF(month, '2026-07-01', '2026-07-31') = 0

DATEDIFF(month, '2026-07-31', '2026-08-01') = 1
```

functions

```
DATEPART(datepart, date) = number

DATENAME(datepart, date) = name

DATEADD(datepart, count, from date) = to date

DATEDIFF(datepart, from date, to date) = count
```

```
DATEDIFF(day, '2026-08-30', '2026-09-03') = 4

DATEDIFF(month, '2026-07-01', '2026-07-31') = 0

DATEDIFF(month, '2026-07-31', '2026-08-01') = 1

DATEDIFF(month, '2024-06-12', '2026-08-08') = 26
```

functions

```
DATEPART( datepart , date ) = number

DATENAME( datepart , date ) = name

DATEADD( datepart , count , from date ) = to date

DATEDIFF( datepart , from date , to date ) = count

DATETRUNC( datepart , date ) = truncated date
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

DATETRUNC(datepart , date) = truncated date

```
DATETRUNC(month, '2026-08-22') = '2026-08-01'
```

functions

```
DATEPART( datepart , date ) = number

DATENAME( datepart , date ) = name

DATEADD( datepart , count , from date ) = to date

DATEDIFF( datepart , from date , to date ) = count

DATETRUNC( datepart , date ) = truncated date
```

```
DATETRUNC(month, '2026-08-22') = '2026-08-01'

DATETRUNC(year, '2026-08-26') =
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

DATETRUNC(datepart , date) = truncated date

```
DATETRUNC(month, '2026-08-22') = '2026-08-01'

DATETRUNC(year, '2026-08-26') = '2026-01-01'
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

DATETRUNC(datepart , date) = truncated date

```
DATETRUNC(month, '2026-08-22') = '2026-08-01'

DATETRUNC(year, '2026-08-26') = '2026-01-01'

DATETRUNC(quarter, '2026-08-26') =
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

DATETRUNC(datepart , date) = truncated date

```
DATETRUNC(month, '2026-08-22') = '2026-08-01'

DATETRUNC(year, '2026-08-26') = '2026-01-01'

DATETRUNC(quarter, '2026-08-26') = '2026-07-01'
```

functions

```
DATEPART(datepart, date) = number

DATENAME(datepart, date) = name

DATEADD(datepart, count, from date) = to date

DATEDIFF(datepart, from date, to date) = count

DATETRUNC(datepart, date) = truncated date

DATE_BUCKET(datepart, 1, date) = "truncated" date
```

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

DATETRUNC(datepart , date) = truncated date

DATE_BUCKET(datepart , 1, date) = "truncated" date

Basically the same result.

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

DATETRUNC(datepart , date) = truncated date

DATE_BUCKET(datepart , 1, date) = "truncated" date

DATE_BUCKET(datepart , count , date) = first date of bucket

date	3 -day bucket
2025-12-02	2025-12-01
2025-12-03	
2025-12-04	2025-12-04
2025-12-05	
2025-12-06	
2025-12-07	2025-12-07
2025-12-08	
2025-12-09	
2025-12-10	2025-12-10
2025-12-11	
2025-12-12	
2025-12-13	2025-12-13
2025-12-14	
2025-12-15	
2025-12-16	2025-12-16

functions

DATEPART(datepart , date) = number

DATENAME(datepart , date) = name

DATEADD(datepart , count , from date) = to date

DATEDIFF(datepart , from date , to date) = count

DATETRUNC(datepart , date) = truncated date

DATE_BUCKET(datepart , 1, date) = "truncated" date

DATE_BUCKET(datepart , count , date) = first date of bucket

DATE_BUCKET(datepart , count , date , start date) = first date of bucket

functions

```
DATEPART( datepart , date ) = number

DATENAME( datepart , date ) = name

DATEADD( datepart , count , from date ) = to date

DATEDIFF( datepart , from date , to date ) = count

DATETRUNC( datepart , date ) = truncated date

DATE_BUCKET( datepart , 1, date ) = "truncated" date

DATE_BUCKET( datepart , count , date ) = first date of bucket

DATE_BUCKET( datepart , count , date , start date ) = first date of bucket
```

start date → 2025-12-05

date	3 -day bucket
2025-12-02	2025-12-02
2025-12-03	
2025-12-04	
2025-12-05	2025-12-05
2025-12-06	
2025-12-07	
2025-12-08	2025-12-08
2025-12-09	
2025-12-10	
2025-12-11	2025-12-11
2025-12-12	
2025-12-13	
2025-12-14	2025-12-14
-15	
-16	

functions

takeaways

- DATEDIFF() returns the difference of the date part, which may not be what you would always expect.

functions

takeaways

- DATEDIFF() returns the difference of the date part, which may not be what you would always expect.
- DATENAME() depends on your language settings

functions

takeaways

- DATEDIFF() returns the difference of the date part, which may not be what you would always expect.
- DATENAME() depends on your language settings
- DATETRUNC() and DATE_BUCKET() are similar, but DATE_BUCKET() is more powerful.

meaningful conversions

meaningful conversions

But seriously, though, what even is 10/11/12?

meaningful conversions

Demo 2

meaningful conversions

takeaways

- Don't rely on DATEFORMAT or SET LANGUAGE

meaningful conversions

takeaways

- Don't rely on DATEFORMAT or SET LANGUAGE
- Use ODBC notation when hard-coding date/time strings

meaningful conversions

takeaways

- Don't rely on DATEFORMAT or SET LANGUAGE
- Use ODBC notation when hard-coding date/time strings
- When possible, always specify the style with CONVERT()

meaningful conversions

takeaways

- Don't rely on DATEFORMAT or SET LANGUAGE
- Use ODBC notation when hard-coding date/time strings
- When possible, always specify the style with CONVERT()
  - The size of the varchar truncates the output

meaningful conversions

takeaways

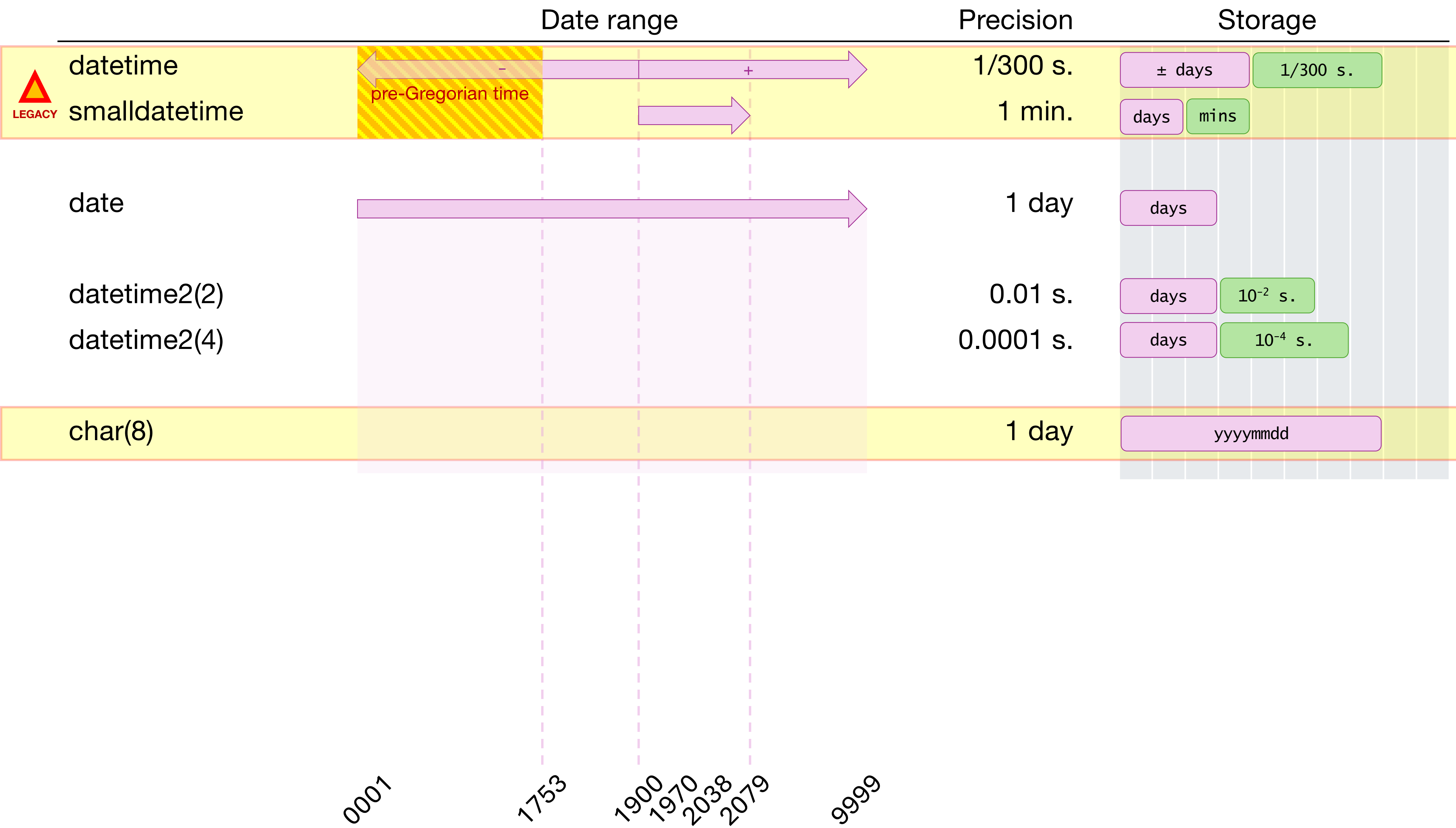
- Don't rely on DATEFORMAT or SET LANGUAGE
- Use ODBC notation when hard-coding date/time strings
- When possible, always specify the style with CONVERT()
  - The size of the varchar truncates the output
  - 121 is yyyy-mm-dd
  - 112 is yyyymmdd

meaningful conversions

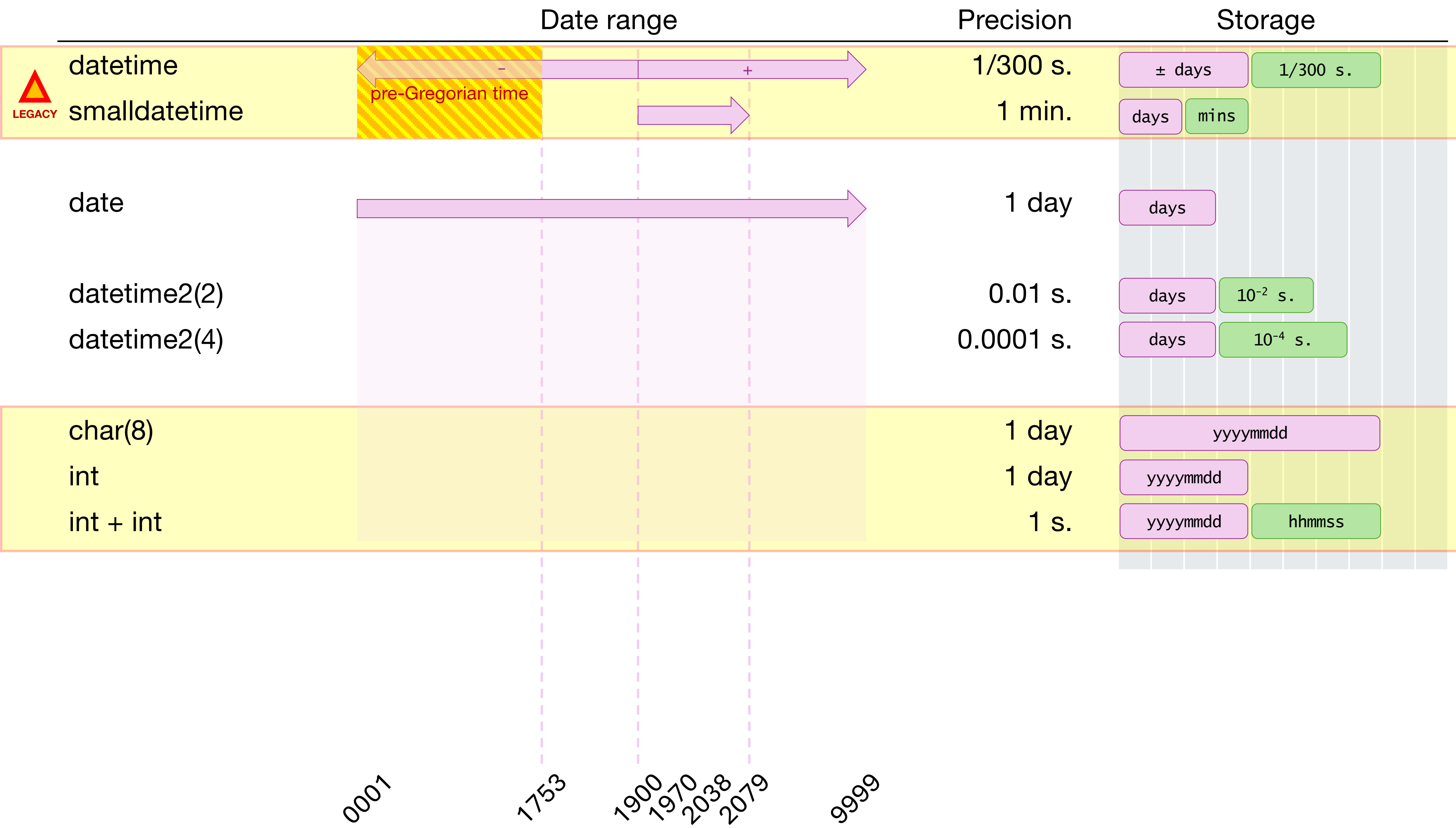
takeaways

- Don't rely on DATEFORMAT or SET LANGUAGE
- Use ODBC notation when hard-coding date/time strings
- When possible, always specify the style with CONVERT()
  - The size of the varchar truncates the output
  - 121 is yyyy-mm-dd
  - 112 is yyyymmdd
- Use TRY_CAST() or TRY_CONVERT() when data quality is unreliable

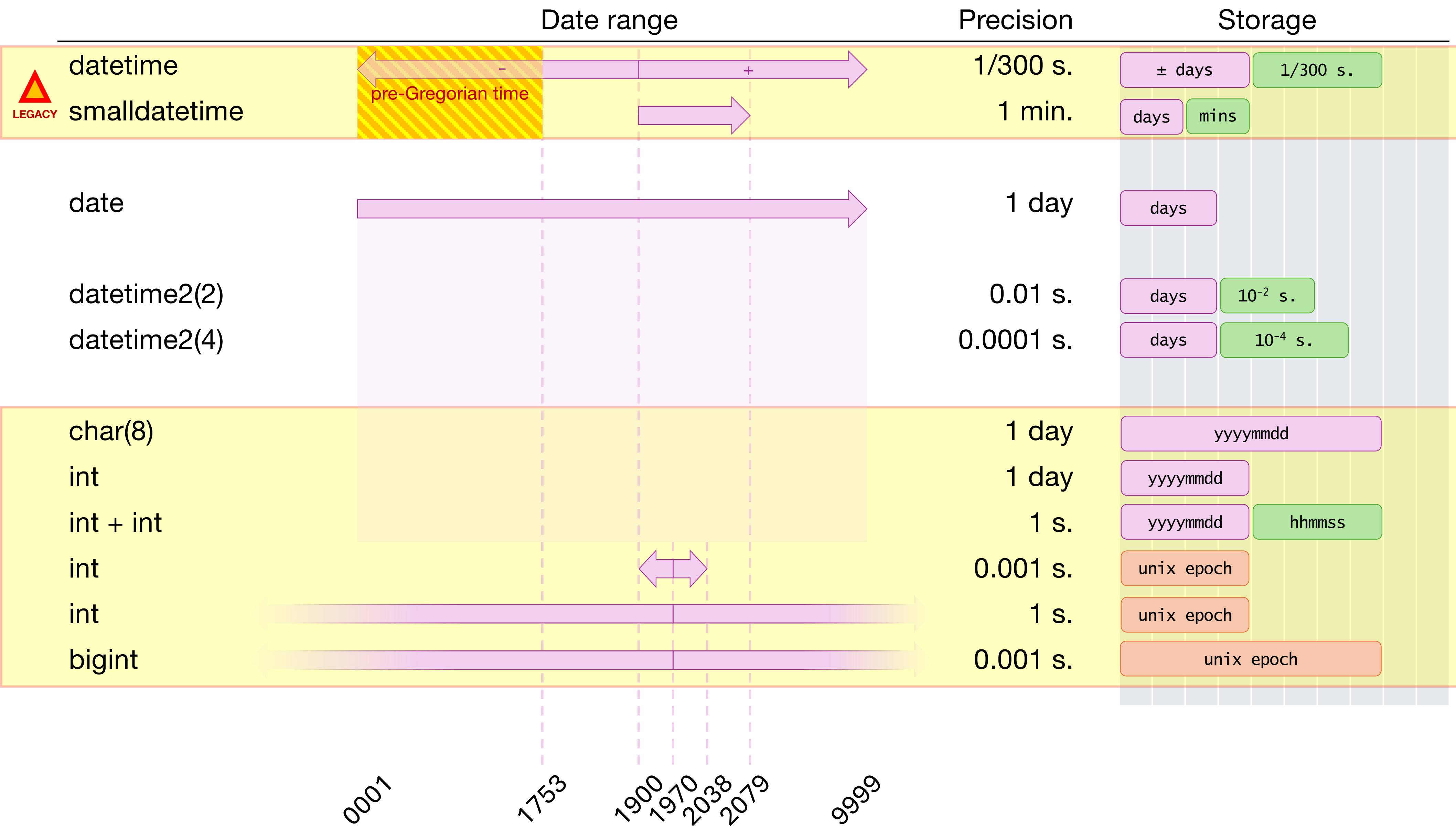
not your type



Date range
Precision
Storage
LEGACY
datetime
smalldatetime
pre-Gregorian time
1/300 s.
1 min.
± days
1/300 s.
days
mins
date
1 day
days
datetime2(2)
datetime2(4)
0.01 s.
0.0001 s.
days
10-2 s.
days
10-4 s.
char(8)
1 day
yyyymmdd
0001
1753
1900
1970
2038
2079
9999



Date range
Precision
Storage
datetime
smalldatetime
LEGACY
pre-Gregorian time
–
+
1/300 s.
1 min.
± days
1/300 s.
days
mins
date
1 day
days
datetime2(2)
datetime2(4)
0.01 s.
0.0001 s.
days
10⁻² s.
days
10⁻⁴ s.
char(8)
int
int + int
1 day
1 day
1 s.
yyyymmdd
yyyymmdd
yyyymmdd
hhmmss
0001
1753
1900
1970
2038
2079
9999



Date range
Precision
Storage
LEGACY
datetime
smalldatetime
pre-Gregorian time
1/300 s.
1 min.
± days
1/300 s.
days
mins
date
1 day
days
datetime2(2)
0.01 s.
days
10^-2 s.
datetime2(4)
0.0001 s.
days
10^-4 s.
char(8)
1 day
yyyymmdd
int
1 day
yyyymmdd
int + int
1 s.
yyyymmdd
hhmmss
int
0.001 s.
unix epoch
int
1 s.
unix epoch
bigint
0.001 s.
unix epoch
0001
1753
1900
1970
2038
2079
9999

not your type

Demo 4

not your type

takeaways

- Store the *data*, not the format.

not your type

takeaways

- Store the *data*, not the format.
- Non-date types
  - don't support date/time functions,

not your type

takeaways

- Store the *data*, not the format.
- Non-date types
  - don't support date/time functions,
  - use more storage,

not your type

takeaways

- Store the *data*, not the format.
- Non-date types
    - don't support date/time functions,
    - use more storage,
    - don't correctly validate dates

not your type

takeaways

- Store the *data*, not the format.
- Non-date types
  - don't support date/time functions,
  - use more storage,
  - don't correctly validate dates
- … but they can allow for some "special" dates, like '2026-01-00' or '2026-13-01' or '24:15:00'.

indexing and sargability

Demo 5

indexing and sargability

takeaways

- Convert and compute your *inputs*, not the column.

indexing and sargability

takeaways

- Convert and compute your *inputs*, not the column.
- Use "half-open" intervals: `WHERE @from<=d AND d<@to`

indexing and sargability

takeaways

- Convert and compute your *inputs*, not the column.
- Use "half-open" intervals: `WHERE @from<=d AND d<@to`
- The optimizer understands that `DATETRUNC()` and `DATE_BUCKET()` maintain the sort order,

indexing and sargability

takeaways

- Convert and compute your *inputs*, not the column.
- Use "half-open" intervals: `WHERE @from<=d AND d<@to`
- The optimizer understands that `DATETRUNC()` and `DATE_BUCKET()` maintain the sort order,
  - … but not for `YEAR()`, `MONTH()`, `DATEPART()`.

utc & time zones

Demo 6

utc & time zones

takeaways

- SWITCHOFFSET() and AT TIME ZONE are similar, but
  - SWITCHOFFSET() only adds or subtracts hours,
  - AT TIME ZONE handles daylight savings time.

week sauce

Demo 7

week sauce

takeaways

- "week" uses DATEFIRST to determine the start of a week

week sauce

takeaways

- "week" uses DATEFIRST to determine the start of a week
- "iso_week" *always* start on Mondays

week sauce

takeaways

- "week" uses DATEFIRST to determine the start of a week
- "iso_week" *always* start on Mondays
- January 1st is always week 1 with "week"

week sauce

takeaways

- "week" uses DATEFIRST to determine the start of a week
- "iso_week" *always* start on Mondays
- January 1st is always week 1 with "week"
- "iso_week" can start with *last year's* week 52 or 53

week sauce

takeaways

- "week" uses DATEFIRST to determine the start of a week
- "iso_week" *always* start on Mondays
- January 1st is always week 1 with "week"
- "iso_week" can start with *last year's* week 52 or 53
- For ISO week years, use the week bucket → compute the year.

nightmare fuel – *and some friendly advice*

Demo 8

questions?

Download slides and scripts:

github.com/sqlsunday/presentations

feed me

Good dates, bad dates: Mastering your T-SQL date and time datatypes

some casual reading

Connor Cunningham on datetime: https://www.sqlskills.com/blogs/conor/1753-datetime-and-you/
Hannah Vernon on storage: https://www.sqlserverscience.com/internals/how-sql-server-stores-datetime/, .../how-sql-server-stores-datetime2/
Peter Larsson on storage: https://weblogs.sqlteam.com/peterl/2010/12/15/the-internal-storage-of-a-datetimeoffset-value/
Randolph West on storage: https://bornsql.ca/blog/how-sql-server-stores-data-types-datetime-date-time-and-datetime2/
Itzik Ben-Gan on DATE_BUCKET(): https://sqlperformance.com/2022/10/t-sql-queries/date-bucket-datetrunc-improve-time-based-grouping
Date and time style (SQL Server docs): https://learn.microsoft.com/en-us/sql/t-sql/functions/cast-and-convert-transact-sql?view=sql-server-ver17#date-and-time-styles